16: Transaction in kafka (Consumer)

Monday, 30 March 2026 7:11 PM

Broker has the records, and below are the 3 possible use cases:

1. Transaction status : ONGOING

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875887784 size: 243 magic: 2 compresscodec: snappy crc: 1578994549 isvalid: true
| offset: 0 CreateTime: 1774875887784 keySize: 4 valueSize: 113 sequence: 0 headerKeys: [__TypeId__] key: O-33 payload: {"orderId":"O-33","customerId":"CUST-2","productId"
:"PROD-3","quantity":1,"totalAmount":99.99,"status":"CREATED"}
```

There is no Control Record for it yet, so means its still in ONGOING state.

2. Transaction status : COMMIT

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875888100 size: 216 magic: 2 compresscodec: snappy crc: 2158642178 isvalid: true
| offset: 0 CreateTime: 1774875888100 keySize: 8 valueSize: 82 sequence: 0 headerKeys: [__TypeId__] key: PAY-O-33 payload: {"paymentId":"PAY-O-33","customerId":"CUST-2","t
otalAmount":99.99,"status":"PAID"}
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH
orizonMs: OptionalLong.empty position: 216 CreateTime: 1774875888151 size: 78 magic: 2 compresscodec: none crc: 3247449494 isvalid: true
| offset: 1 CreateTime: 1774875888151 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [] endTxnMarker: COMMIT coordinatorEpoch: 0
```

3. Transaction status : ABORT

Similarly like COMMIT, we have control record entry for ABORT.

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774901083813 size: 243 magic: 2 compresscodec: snappy crc: 172206083 isvalid: true
| offset: 0 CreateTime: 1774901083813 keySize: 4 valueSize: 113 sequence: 0 headerKeys: [__TypeId__] key: O-33 payload: {"orderId":"O-33","customerId":"CUST-2","productId"
:"PROD-3","quantity":1,"totalAmount":99.99,"status":"CREATED"}
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH
orizonMs: OptionalLong.empty position: 243 CreateTime: 1774901084304 size: 78 magic: 2 compresscodec: none crc: 3109029820 isvalid: true
| offset: 1 CreateTime: 1774901084304 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [] endTxnMarker: ABORT coordinatorEpoch: 0
```

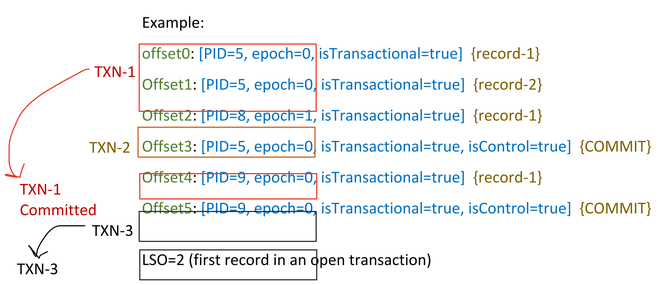
Concept of LSO (Last Stable Offset):

LSO -> offset of the first message that belongs to an OPEN (ongoing) Transaction.

A "read_committed" consumer CAN NOT read past the LSO. Even if there are committed or non transaction records.

Example:

Topic: Order-events
Partition: P2
Logs:



Committed	Maintained by broker and keep it in-memory.

Consumer

spring.kafka.consumer.properties.isolation.level=read_committed / read_uncommitted (default)

read_uncommitted (default)

Reads everything expect *control records (isControl=true)*
- Ongoing transactional records
- Committed transactional records
- Aborted transactional records
- Non-transaction records

For above example:

Consumer will read:
- Offset0 - part of committed txn-1
- Offset1 - part of committed txn-1
- Offset2 - Open (ongoing) txn-2
- Offset4 - part of committed txn-3

Skips:
- Offset3 - Control record for Txn-1
- Offset5 - Control record for Txn-3

read_committed

Reads:
- Committed transactional records
- Non-transactional records
- Skips Aborted transactional records
- Stops at LSO
- Skips control records(isControl = true) (Broker itself do not return it)

For above example:

Consumer will read:
- Offset0 - part of committed txn
- Offset1 - part of committed txn

LSO = 2, so consumer can not read it, as it denotes this Txn is still open.

Notice: even though Txn-3 is committed but consumer can not read it, because of LSO, this ensures that consumer never sees partially completed transactions preserving both ordering and atomicity guarantees.

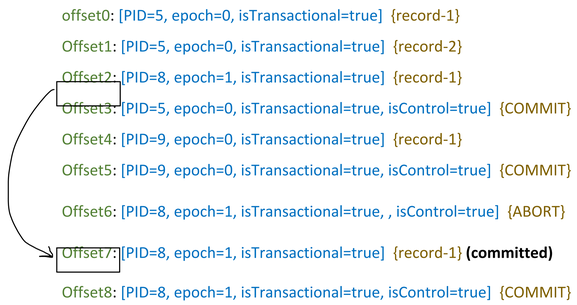
How Consumer with "read_committed" skips ABORTED records?

This is interesting:

Aborted records are not filtered at Broker layer. Its filtered at Consumer layer.

But question is:

How consumer know, which offsets need to be skip ?



Here for "read_committed" consumer, broker will return:

- offset0: [PID=5, epoch=0, isTransactional=true] {record-1} **(committed)**
- Offset1: [PID=5, epoch=0, isTransactional=true] {record-2} **(committed)**
- Offset2: [PID=8, epoch=1, isTransactional=true] {record-1} **(Aborted)**
- Offset4: [PID=9, epoch=0, isTransactional=true] {record-1} **(committed)**
- Offset7: [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)**

Skips Control records:

- Offset3: [PID=5, epoch=0, isTransactional=true, isControl=true] {COMMIT}
- Offset5: [PID=9, epoch=0, isTransactional=true, isControl=true] {COMMIT}
- Offset6: [PID=8, epoch=1, isTransactional=true, , isControl=true] {ABORT}
- Offset8: [PID=8, epoch=1, isTransactional=true, isControl=true] {COMMIT}

Now how consumer filter out this Aborted record?

Also Notice: there are 2 records with same PID and epoch

- offset0: [PID=5, epoch=0, isTransactional=true] {record-1} **(committed)**
- Offset1: [PID=5, epoch=0, isTransactional=true] {record-2} **(committed)**
- Offset2: [PID=8, epoch=1, isTransactional=true] {record-1} **(Aborted)**
- Offset4: [PID=9, epoch=0, isTransactional=true] {record-1} **(committed)**
- Offset7: [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)**

So only Offset2 should be skipped, not Offset7.

Answer is provided by Broker itself:

For every transaction, which is ABORTED. Broker maintains a separate index file:

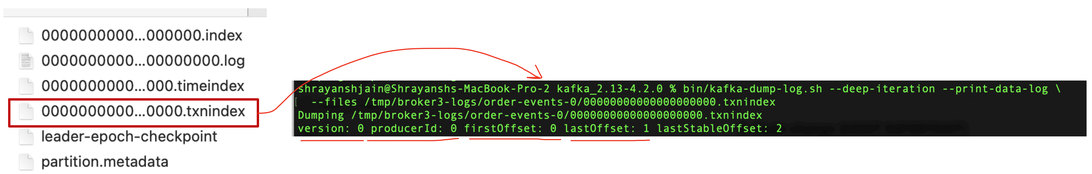
ex:
00000000000000000000.txnindex

```
[
    {
        PID: 8,
        Epoch: 1,
        First offset Index: 2,
        Control record offset (last offset): 6
    }
]
```

So, Consumer uses this file to know, what all records are aborted and skips them.

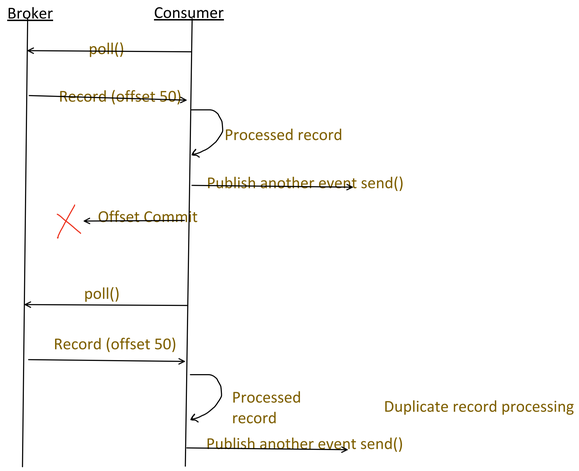
- offset0: [PID=5, epoch=0, isTransactional=true] {record-1} **(committed)** ← *Accepted, PID and epoch do not match* ✓
- Offset1: [PID=5, epoch=0, isTransactional=true] {record-2} **(committed)** ← *Accepted, PID and epoch do not match* ✓
- Offset2: [PID=8, epoch=1, isTransactional=true] {record-1} **(Aborted)** ← *Skipped, as PID=8, epoch=1 and offset 2 is within range [2-6]* ✗
- Offset4: [PID=9, epoch=0, isTransactional=true] {record-1} **(committed)** ← *Accepted, PID and epoch do not match* ✓
- Offset7: [PID=8, epoch=1, isTransactional=true] {record-1} **(committed)** ← *Accepted, as PID=8, epoch=1 but offset 7 is not in range [2-6]* ✓

know



So this one problem we discussed exists at Consumer side.

Duplicate Processing (Read-Process-Write) - Consumer side problem



This is core EOS (Exactly One Semantics) problem:

read -> process -> write -> commit offset

The above problem happens because: Write and Commit Offset is a separate step. So if any crash happens, could result in:

- Duplicate writes

That's where the name comes: EOS (Exactly once), no duplicates.

So in this use-case :

Read -> Process -> Write -> Commit offset

EOS achieved = read_committed Consumer + Idempotency + Transaction

In simple term, with the help of EOS, write and commit offset step is made atomic.

Read -> Process -> Write -> Commit offset

Any failure, write and commit offset will not happen.

Implementation:

application.properties

```
.
. (other consumer properties)
.

spring.kafka.consumer.properties.isolation.level=read_committed
spring.kafka.consumer.enable-auto-commit=false


spring.kafka.producer.transaction-id-prefix=order-serv-
spring.kafka.producer.acks=all
spring.kafka.producer.enable-idempotence=true
```

```
@Component
public class OrderEventListener {

  @Autowired
  private KafkaTemplate<String, Object> kafkaTemplate;

  @KafkaListener(topics = "order-events")
  public void process(ConsumerRecord<String, Object> record) {

    kafkaTemplate.executeInTransaction(ops -> {

      // 1. READ happened and a record is received in this method
        String key = record.key();
        Object value = record.value();


      // 2. PROCESS, any business logic here if you want


      // 3. WRITE (to another topic)
        ops.send("payment-events", key, result);

      // 4. SEND OFFSET TO TRANSACTION, we are just buffering the offsets which need to be Committed when this txn is committed.
        Map<TopicPartition, OffsetAndMetadata> offsets = new HashMap<>();
        offsets.put(
            new TopicPartition(record.topic(), record.partition()),
            new OffsetAndMetadata(record.offset() + 1)
        );

        ops.sendOffsetsToTransaction(offsets, "order-group");

        return null;
      });
    }
}
```